Ex-6
10/9/2026

Python Implementation Of Hamming Code Concept

Error Correction at Data Link Layer (AIM):

→ Hamming code is a set of error-correction codes that can be used to detect and correct the errors when the data is transmitted from the sender to the receiver. To implement error detection & correction using Hamming code technique at Data link Layer.

SENDER PROGRAM:-

→ Takes text input
→ Converts text to binary
→ Divides binary data into 4-bit groups
→ Adds 3 redundant parity bits using Hamming code.
→ Saves the encoded data in 'channel.txt'

CODE:-

```
def hamming_encode(data):
```

CODE:-

```
def hamming_encode(data):
    encoded = ""

    for i in range(0, len(data), 4):
        d = data[i:i+4]

        if len(d) < 4:
            d = d.ljust(4, '0')

        d1, d2, d3, d4 = map(int, d)

        p1 = d1 ^ d2 ^ d4
        p2 = d1 ^ d3 ^ d4
        p4 = d2 ^ d3 ^ d4

        code = str(p1) + str(p2) + str(d1) + str(p4) + str(d2) +
                                                    str(d3) + str(d4)

        encode += code

    return encoded
text = input("Enter text: ")
binary = ''.join(format(ord(c), '08b') for c in text)
print("Binary data:", binary)
encoded_data = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
print("Hamming encoded data:", encoded_data)
Print("Data saved to channel.txt")
```

RECEIVER PROGRAM:-

→ Reads data from channel.txt
→ Checks every 7-bit hamming block
→ Finds the error position using syndrome.
→ Corrects the errors & removes parity bits
→ Converts binary back to ASCII text.

CODE:-

```
def hamming_decode(data):
    corrected = ""
    error_found = False

    for i in range(0, len(data), 7):
        block = list(map(int, data[i:i+7]))
        if len(block) < 7:
            continue
        p1, p2, d1, p4, d2, d3, d4 = block
        s1 = p1 ^ d1 ^ d2 ^ d4
        s2 = p2 ^ d1 ^ d3 ^ d4
        s4 = p4 ^ d2 ^ d3 ^ d4
        error_position = s1 + (s2 * 2) + (s4 * 4)
```

```
        if error_position != 0:
            error_found = True
            actual_position = i + error_position
            print("Error detected at bit position:", actual_position)
            block[error_position - 1] ^= 1
            print("Error corrected at bit position:", actual_position)

        corrected += str(block[2])
        corrected += str(block[4])
        corrected += str(block[5])
        corrected += str(block[6])

    return corrected, error_found

with open("channel.txt", "r") as file:
    channel_data = file.read().strip()
print("Data received from channel:")
print(channel_data)

binary_data, error_found = hamming_decode(channel_data)
text = ""
```

```
for i in range (0, len(binary_data), 8):
    byte = binary_data [i:i+8]
    if len (byte) == 8:
        text += chr (int (byte, 2))
Print ("\n Corrected binary data: ")
Print (binary_data)
if error_found:
    print ("\n Error was detected and corrected. ")
else:
    print ("\n No error detected. ")
print ("Received text: ", text
```

INPUT:-

Enter the text: HELLO

OUTPUT:-

Binary data: 0100100001000101010010001001100010011

Hamming encoded data:- 1001100111000100110001001010111000111100100111111

Data saved to channel.txt

RESULT:-

Thus the Hamming code was successfully implemented to detect and correct errors in the transmitted data.